Compartilhaê na Arquitetura CORBA

Mapeamento dos componentes da arquitetura CORBA (OMG) para a plataforma distribuída de compartilhamento de alimentos excedentes

Integrante	RA
Vitor Aparecido Lavezzo	G7870-2
Vinícius Araújo de Freitas	F3520C-5
Kayky Azevedo da Rocha	F3511B-8

Introdução

O **Compartilhaê** conecta estabelecimentos que possuem alimentos excedentes (restaurantes, mercados e padarias, os *doadores*) a ONGs que os recebem. O sistema é distribuído: cada instância do backend é um **nó regional** independente (Limeira, Campinas e São Paulo), identificado por `NODE_ID`. Os nós se comunicam por eventos publicados num Redis Stream e por um barramento Socket.IO compartilhado via Redis.

O Compartilhaê não usa CORBA literalmente. Não há IDL da OMG nem protocolo IIOP: a comunicação é feita com HTTP/JSON (REST), WebSocket e Redis. Mesmo assim, cada papel definido pela arquitetura CORBA tem um equivalente direto no projeto. O objetivo do CORBA é que o cliente invoque operações em objetos remotos **sem conhecer onde nem como eles estão implementados**, e o Compartilhaê busca o mesmo resultado. Este documento responde a cada tópico da especificação mostrando qual parte do sistema exerce aquele papel.

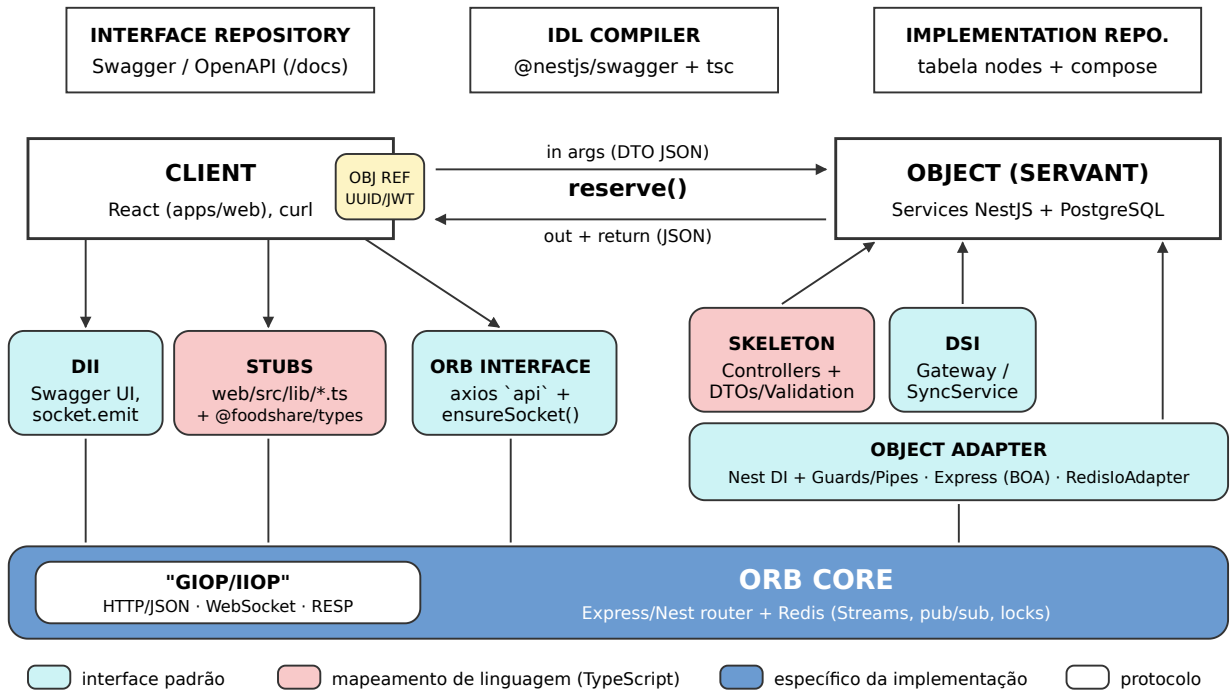


Figura 1: A figura da família CORBA redesenhada com os componentes equivalentes do Compartilhaê.

5.1. Clientes

No Compartilhaê, os clientes são os programas que invocam operações sobre as "implementações de objeto" do backend. O principal é o **frontend React** (`apps/web`), usado por três papéis: **doador** (publica listas de doação), **ONG** (busca, reserva e confirma retirada) e **administrador** (aprova organizações e monitora nós e sincronização). Qualquer cliente HTTP, como `curl` ou a própria Swagger UI, também atua como cliente.

A transparência exigida pelo CORBA está presente. Quando a ONG reserva uma doação, o componente React chama uma função local, `reserveDonationItems(donationListId, items)`, e só precisa conhecer essa interface. Ele não sabe qual nó (Limeira, Campinas ou São Paulo) atenderá a requisição, como o token é anexado ou renovado, nem que a reserva trava a linha no PostgreSQL com `SELECT ... FOR UPDATE`. Para o cliente, a chamada parece local.

Assim como no CORBA, a invocação pode ser feita de duas formas:

- **Estática (via stub):** pelas funções tipadas de `apps/web/src/lib/*.ts` (ver 5.7);
- **Dinâmica (via DII):** montando a requisição em tempo de execução a partir da descrição da API publicada em `/docs`, ou emitindo eventos Socket.IO pelo nome (ver 5.5).

As referências a objetos que o cliente manipula são os identificadores (UUIDs) das entidades, como em `/donations/:id`, e o **código de retirada** apresentado presencialmente, que referencia uma reserva específica.

5.2. Implementação de Objetos

As implementações de objeto são os **serviços de domínio do NestJS**, que contêm o código real das operações expostas pela interface:

"Objeto" (servant)	Operações	Dados internos
`DonationsService`	criar, listar, editar e cancelar listas de doação	tabelas `donation_lists` e itens
`ReservationsService`	reservar (total/parcial), cancelar, confirmar retirada	`reservations`, código de retirada
`OrganizationsService`	cadastrar e aprovar doadores/ONGs	`organizations`, endereço, CNPJ
`ChatService`, `NotificationsService`	mensagens e notificações	`messages`, `notifications`
`SyncService`, `NodesService`	publicar/consumir eventos, heartbeat	`sync_events`, `nodes`

O estado dos objetos fica persistido no **PostgreSQL** (via Prisma), compartilhado pelos nós. O código dos métodos está nos arquivos `*.service.ts`. Os **procedimentos de ativação e desativação** definidos na criação do objeto correspondem aos ganchos de ciclo de vida do NestJS: `NodesService.onModuleInit()` registra o nó e inicia o heartbeat a cada 15 s, e `onModuleDestroy()` o encerra. O `SyncService` inicia e encerra o laço de leitura do stream da mesma maneira.

```
async onModuleInit() {            // ativação
  await this.upsertSelf();        // registra o nó como ONLINE
  this.heartbeatTimer = setInterval(() => this.upsertSelf(), 15_000);
}
onModuleDestroy() {               // desativação
  clearInterval(this.heartbeatTimer);
}
```

5.3. ORB

O papel do ORB, que é transportar de forma transparente a invocação do cliente até a implementação e devolver o resultado, é dividido no Compartilhaê entre três camadas:

- **Núcleo HTTP (Express + roteador do NestJS):** localiza a implementação certa para cada requisição. `POST /donations/:id/reserve` é roteado até o método do controller e deste ao `ReservationsService`. O resultado volta serializado em JSON.
- **Socket.IO + Redis:** entrega eventos em tempo real (mensagens de chat, notificações, `data:changed`) a qualquer cliente, **independentemente do nó ao qual ele está conectado**, graças ao `RedisIoAdapter`.
- **Redis Streams (`SyncService`):** comunicação entre nós. Cada transição de estado relevante (`DONATION_CREATED`, `DONATION_RESERVED`, `PICKUP_CONFIRMED` etc.) é publicada com `XADD` e consumida pelos demais nós com `XREAD`. Cada nó guarda seu cursor, o que permite retomar a leitura após uma queda.

O equivalente ao protocolo **GIOP/IIOP** é a combinação HTTP/JSON (cliente-servidor), protocolo Socket.IO sobre WebSocket (tempo real) e RESP, o protocolo do Redis (entre nós). Como o CORBA prevê, o programador do frontend não escreve nenhum detalhe de baixo nível de rede: serialização, cabeçalhos, renovação de token e reconexão ficam encapsulados.

A tarefa de "ativar o objeto, caso não esteja ativo" aparece na **escalabilidade horizontal**. Um novo nó é apenas um novo container com outro `NODE_ID` (`docker compose --profile node-campinas up -d api-campinas`), e o `LeaderLockService` (lock `SET NX EX` no Redis) garante que tarefas agendadas rodem em um único nó por ciclo.

5.4. Interface ORB

A interface ORB é a biblioteca local que desacopla a aplicação dos detalhes do "ORB". No Compartilhaê, esse papel cabe a dois módulos do frontend:

- **`apps/web/src/lib/api.ts`:** a instância `axios` compartilhada. Define a URL base do nó, anexa o `Authorization: Bearer` a toda requisição, trata o erro 401 renovando o token de forma transparente (uma única renovação concorrente) e padroniza o formato de erro (`getApiErrorMessage`).
- **`apps/web/src/lib/socket.ts`:** `ensureSocket()` abre, reabre (após renovação de token) e mantém a conexão de tempo real, preservando os listeners registrados.

O CORBA cita a **conversão de referências a objetos em strings e vice-versa** como uma das funções dessa interface. No Compartilhaê, toda referência já trafega como string: os UUIDs das entidades nas URLs e o **JWT**, que codifica a identidade e o papel do usuário em uma string assinada e é decodificado de volta pelo `JwtStrategy` no servidor. A **criação de**

listas de argumentos para a DII corresponde à montagem genérica de `params`/`body` que o axios faz para qualquer rota.

5.5. Interface de Invocação Dinâmica (DII)

A DII permite invocar operações descobertas em tempo de execução, sem stub compilado. No Compartilhaê existem três formas disso:

- **Swagger UI (`/docs`):** lê a descrição OpenAPI da API em tempo de execução, descobre as operações (`POST /donations`, `POST /donations/:id/reserve`, ...) e seus parâmetros, monta a requisição e a envia. Nenhum código do cliente foi gerado para isso: é invocação puramente dinâmica.
- **Chamadas genéricas** `api.request({ method, url, data })` ou `curl`, nas quais a operação é escolhida por string.
- **Eventos Socket.IO por nome:** `socket.emit('message:send', payload)`, em que a "operação" é uma string resolvida no servidor.

Os dois modos de chamada que o CORBA associa à DII também existem:

- **Síncrona diferida não-bloqueante:** toda chamada HTTP retorna uma `Promise`. O TanStack Query dispara a requisição e a interface continua respondendo; o resultado é recebido depois (envio e recebimento separados).
- **Unidirecional (só envio):** `socket.emit('conversation:open', ...)` e `conversation:close` não esperam resposta, assim como a publicação `XADD` no stream de sincronização, em que o publicador não aguarda os consumidores.

5.6. Interface de Esqueleto Dinâmico (DSI)

A DSI é o lado servidor da DII: ela permite entregar a uma implementação requisições cujo tipo não é conhecido estaticamente. No Compartilhaê, dois componentes fazem isso:

- **`RealtimeGateway`** (`apps/api/src/modules/realtime/realtime.gateway.ts`): recebe mensagens WebSocket genéricas e as despacha pelo nome do evento com `@SubscribeMessage('message:send')`, `'conversation:open'` e `'conversation:close'`, sem rota HTTP tipada.
- **Consumidor do `SyncService`:** lê do Redis Stream entradas que são apenas um campo `data` com JSON. O tipo do evento (`eventType`) só é conhecido em tempo de execução, ao desserializar a mensagem. O consumidor descarta o eco do próprio nó (`originNode === nodeId`) e trata o evento conforme seu tipo.

```
const [, entries] = await reader.xread('BLOCK', BLOCK_MS, 'STREAMS', STREAM_KEY, cursor);
for (const [entryId, fields] of entries) {
  this.handleIncoming(fields[fields.indexOf('data') + 1]);  // tipo descoberto em runtime
}
```

5.7. Stub

Os stubs ficam no cliente e oferecem uma interface estática para criar e enviar as requisições. No Compartilhaê, são os módulos de `apps/web/src/lib/` (`donations.ts`, `reservations.ts`,

`organizations.ts`, `chat.ts`, ...). Cada um expõe funções tipadas que escondem a montagem da chamada HTTP:

```
export async function reserveDonationItems(donationListId: string, items: ReserveItemInput[])
{...}
export async function cancelReservation(reservationId: string) {...}
export async function confirmPickup(pickupCode: string) {...}
```

Os tipos compartilhados entre cliente e servidor (categorias, status de doação e de reserva) vêm do pacote `@foodshare/types` (`packages/types`). Esse pacote cumpre o papel do **mapeamento da IDL para a linguagem** (TypeScript): o mesmo contrato é compilado nos dois lados. No CORBA, o stub é gerado pelo compilador IDL. No Compartilhaê, ele é escrito à mão sobre os tipos compartilhados, mas poderia ser gerado automaticamente a partir do documento OpenAPI (com `openapi-generator`, por exemplo), o que completaria a analogia.

5.8. Skeleton

O skeleton é o par do stub no servidor: é a interface pela qual o método recebe as requisições. No Compartilhaê, esse papel é dos **controllers do NestJS** com seus **DTOs**. O controller declara, por decorators, a operação exposta. O DTO, validado pelo `ValidationPipe` global, faz o equivalente ao *unmarshalling*: converte o JSON recebido em um objeto tipado, rejeita campos desconhecidos (`forbidNonWhitelisted`) e só então chama a implementação.

```
@Controller('donations')
export class DonationsController {
  @Post() @Roles(UserRole.DONOR)
  create(@CurrentUser() user, @Body() dto: CreateDonationDto) {
    return this.donationsService.create(user, dto);   // delega ao "servant"
  }
}
```

Stub e skeleton estão vinculados pelo mesmo contrato. A função `createDonation` do frontend envia exatamente o formato que `CreateDonationDto` espera, e ambos se baseiam nos tipos de `@foodshare/types`.

5.9. Adaptador de Objeto

O adaptador de objeto auxilia o ORB no despacho das requisições e na ativação dos objetos, associando as implementações ao ORB. No Compartilhaê, esse papel é do **contêiner de injeção de dependências do NestJS** e da sua cadeia de despacho:

- **Ativação:** o contêiner DI instancia cada serviço como singleton na subida do nó e o injeta nos controllers. É ele que associa o skeleton (controller) ao servant (service).
- **Despacho:** antes de chegar ao objeto, a requisição passa pelo `JwtAuthGuard` (autenticação global, com exceções via `@Public()`), pelo `RolesGuard` (autorização por papel, via `@Roles(...)`), pelo `ValidationPipe`, pelo `DecimalSerializerInterceptor` e pelo `AllExceptionsFilter`, que padroniza os erros.

Assim como o CORBA tem um adaptador padrão presente em todos os ORBs (o **BOA**, *Basic Object Adapter*) e adaptadores especializados, o Compartilhaê também tem:

Tipo CORBA	Equivalente no Compartilhaê

BOA (padrão)	`ExpressAdapter` do NestJS (`NestExpressApplication`), presente em todo nó e responsável por toda a API REST.
Adaptador especializado (tempo real)	`RedisIoAdapter` (`apps/api/src/modules/realtime/redis-io.adapter.ts`), que estende o `IoAdapter` para propagar eventos Socket.IO entre os processos dos nós via Redis pub/sub.
Adaptador persistente (como o OODB)	`PrismaService`, que associa os objetos de domínio às tabelas do PostgreSQL.

Repositórios e compilador IDL

A figura da família CORBA também traz três elementos auxiliares:

- **Repositório de Interfaces:** o documento **OpenAPI** servido em `/docs`. Ele descreve, em tempo de execução, todas as operações, parâmetros e tipos disponíveis, e é essa descrição que viabiliza a DII.
- **Compilador IDL:** o `@nestjs/swagger`, que gera a descrição OpenAPI a partir dos decorators (`@ApiProperty`, `@ApiOperation`), junto com o compilador TypeScript, que verifica o mesmo contrato de `@foodshare/types` no cliente e no servidor.
- **Repositório de Implementações:** a tabela `nodes`, na qual cada nó se registra e envia heartbeat (fica `OFFLINE` após 45 s sem sinal), e o `docker-compose.yml`, que define quais implementações (nós) existem e como iniciá-las. Esse estado é consultado em `GET /nodes` e no painel "Nós & Sincronização".

Resumo do mapeamento

Componente CORBA	Equivalente no Compartilhaê
Cliente	Frontend React (`apps/web`), Swagger UI, curl
Implementação de objetos	Services NestJS + estado no PostgreSQL; ciclo de vida `onModuleInit/Destroy`
ORB / ORB Core	Express + roteador Nest, Socket.IO, Redis (Streams, pub/sub, locks)
GIOP/IIOP	HTTP/JSON, WebSocket (Socket.IO), RESP
Interface ORB	`lib/api.ts` (axios + refresh de token) e `lib/socket.ts`
DII	Swagger UI, `api.request` genérico, `socket.emit`
DSI	`RealtimeGateway` (`@SubscribeMessage`), consumidor do `SyncService`
Stub	`apps/web/src/lib/*.ts` + `@foodshare/types`
Skeleton	Controllers NestJS + DTOs + `ValidationPipe`
Adaptador de objeto (BOA)	DI do NestJS + guards/pipes; `ExpressAdapter` (BOA), `RedisIoAdapter`, `PrismaService`
Repositório de interfaces	OpenAPI em `/docs`
Repositório de implementações	Tabela `nodes` + `docker-compose.yml`

Referência a objeto	UUID da entidade, JWT, código de retirada

Diferenças em relação ao CORBA

- **Interface orientada a recursos, não a objetos remotos:** o Compartilhaê expõe recursos REST e não referências a objetos remotos com estado. Os servants não guardam estado em memória entre chamadas: o estado fica no banco, o que permite que qualquer nó atenda qualquer requisição.
- **Contrato por OpenAPI e TypeScript, não por IDL da OMG:** os stubs são escritos à mão, não gerados.
- **Banco compartilhado:** todos os nós usam o mesmo PostgreSQL. A sincronização por Redis Streams (com `eventId` único, idempotência, retry e catch-up por cursor) já é o mecanismo necessário para uma futura migração para um banco por nó.
- **Consistência:** a reserva usa transação com `SELECT ... FOR UPDATE` (consistência forte; testado em `reservations.concurrency.spec.ts`, em que duas reservas simultâneas resultam em exatamente uma bem-sucedida). Dashboards e relatórios aceitam consistência eventual.